SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

ZenTravel is an AI-powered autonomous travel incident management platform designed to eliminate the manual effort caused by travel disruptions. It acts as a strategic solution to the fragmented and high-stress workflows currently required to resolve travel disruptions such as cancelled or delayed flights.

1.1. Purpose:

This System Analysis Document (SAD) records the technical scope and design decisions behind ZenTravel: an AI-assisted travel disruption recovery and trip planning companion delivered as a React (Vite) single-page application, packaged for Android via Capacitor, with Firebase for identity and booking persistence and Z.AI GLM for structured extraction, ReAct planning, and specialist-agent responses.

1. ***Architecture**:*
    a. *It is built around a rich React SPA that hosts GLM calls and orchestration (business rules, workflow engine, mock tooling). Identity and data use managed Firebase services (Auth, Firestore, Storage as configured), while intelligence and integrations use ILMU, EmailJS, and the exchange-rate API as client-invoked HTTPS endpoints. Vite provides dev-time proxying for the GLM path to address CORS and long "thinking" idle timeouts during development. Mock travel behaviour is implemented in mockTravelAPI, executed in-process so flight, hotel, and compensation-style outcomes are deterministic and simulated for demos.*
2. ***Data Flows:***
    a. *User text and files flow through vision/ document extraction (pdf.js, Tesseract.js) → Brain Master workflow (runZenTravelWorkflow) or specialist agents → UI cards (WorkflowResponseCard, AgentResponseCard). Signed-in users sync profile preferences, bookings, search history, favorites, and reviews with Firestore via listeners and one-off reads/ writes.*
3. ***Model Process:***
    a. *The end-to-end workflow of the core disruption feature: Stage 1 structured extraction (GLM + regex fallback) → validation (clarification pause) → ReAct Reason (tool plan) → Act (mock API execution) → Observe (synthesis) → strategy (recovery narrative), with human approval on sensitive actions and optional EmailJS / mailto for outbound email.*
4. ***Role of Reference:***
    a. *This document is a high-level contract for developers and testers: it describes component boundaries, external dependencies, and data stores so that implementation, QA, and demo preparation stay aligned with the PRD.*

1.2. Background:

© UMHackathon 2026

When flights are delayed, cancelled, or baggage is lost, travellers must coordinate rebooking, accommodation, and claims across opaque policies and channels. ZenTravel unifies narrative and document input with structured planning and mock sourcing in one product surface.

1. *Previous Version:*
    a. *Isolated "chat-only" or "booking-only" demos without unified evidence → workflow → tool execution.*
2. *Changes in Major Architectural Components*
    a. *Brain Master multi-stage engine with ReAct and fallbacks.*
    b. *GLM via ILMU with thinking-model handling (reasoning_content / content, token floor, retries).*
3. *New capabilities that are only introduced in this*
    Document ingestion (PDF, OCR), specialist agents, trip planner PDF export, Firestore-backed bookings and cross-feature navigation.

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Traveller (consumer)	*Uses web or Android app for disruption help, planning, bookings*	*Clear steps, confidence/ disclaimers, no silent automated bookings*
Trip planner	*Builds itineraries, exports PDF*	*Predictable export, template + optional AI copy*
Product / engineering	*Maintains app, GLM prompts, Firebase rules*	*Documented flows, testable mock boundaries, env-based configuration*
QA	*Validates flows and edge cases*	*Defined scope, request/ response shapes for GLM JSON, mock API stability*
Evaluators	*Assesses GLM integration depth*	*Observable stages, not a single opaque chat completion*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web SPA (primary). Android app (Capacitor WebView hosting the same built assets).*
Application style	*React + TypeScript, lazy-loaded pages, in-browser OCR/PDF, GLM client*
Data persistence	*Google Firebase: Auth, Firestore for app data, Storage where used (managed, cloud-hosted services)*
Intelligence	*ILMU: model default* ***ilmu-glm-5.1***

ZenTravel is ***client–external-services****: the React app runs in the browser (or WebView) and talks to Firebase, ILMU, EmailJS, and public HTTP APIs such as exchange rates. Mock travel logic lives in the same bundle, orchestrated from tools.ts/ workflowEngine.ts.*

2.1.2 LLM as Service Layer

The following describes how GLM is integrated as an explicit service layer: prompt construction, context, parsing, and limits.

Aspect	Implementation
a. Prompts sent to GLM	*glmClient.ts posts OpenAI-style messages (system / user / assistant) to VITE_GLM_API_URL or dev /ilmu-api/v1 → .../chat/completions. Staged prompts live in workflowEngine.ts (extraction, tool planning, observe/synthesis, strategy) and specializedAgents.ts (domain JSON contracts). tools.ts holds the ReAct planner system prompt for tool selection.*
b. Context window	*User text is truncated before Stage 1 (8,000*

© UMHackathon 2026

	characters for raw workflow input, 4,000 characters for vision path caps extracted text). Document pipeline uses pdf.js + Tesseract.js, output is text-only into the same extraction path.
c. Receiving, parsing, passing on	*GLM returns JSON in visible content and may use reasoning_content for thinking. Parsers extract JSON (regex or JSON.parse after fence stripping). Results feed validation, tool execution (executeTool from mockTravelAPI), then synthesis and UI state (WorkflowResponseCard).*
d. Token/ input limits	*max_tokens has a floor (1024) and may grow on finish_reason: length retry up to a ceiling (8192). 90 s client timeout. 2 retries on 5xx/timeout/empty body patterns. Truncation is not multi-chunk RAG in current design.*
e. Major API / integration calls	*Browser → Firebase (Auth, Firestore read/write, Storage available).* *Browser → ILMU POST /v1/chat/completions.* *Browser → EmailJS (optional recovery email).* *Browser → Exchange rate API when VITE_EXCHANGE_RATE_API_KEY is set.*
f. Frontend, services, database, external API interaction	*See dependency diagram and sequence below. Firebase stores users, Booking, search_history, favorites, My_Reviews (collection names as used in code). GLM is stateless per request.*
g. Distributed components	*Deployment model: a static web bundle (**Vite build**) plus **Firebase** and **SaaS APIs**. Capacitor packages the same bundle for Android.*

© UMHackathon 2026

2.1.3 Dependency Diagram

The diagram below ties together:

(A) pre-GLM truncation (inputs are not multi-chunk "RAG"-split; long text is cut to a max length before a request is built),

(B) prompt construction per stage,

(C) GLMClient → ILMU request shape,

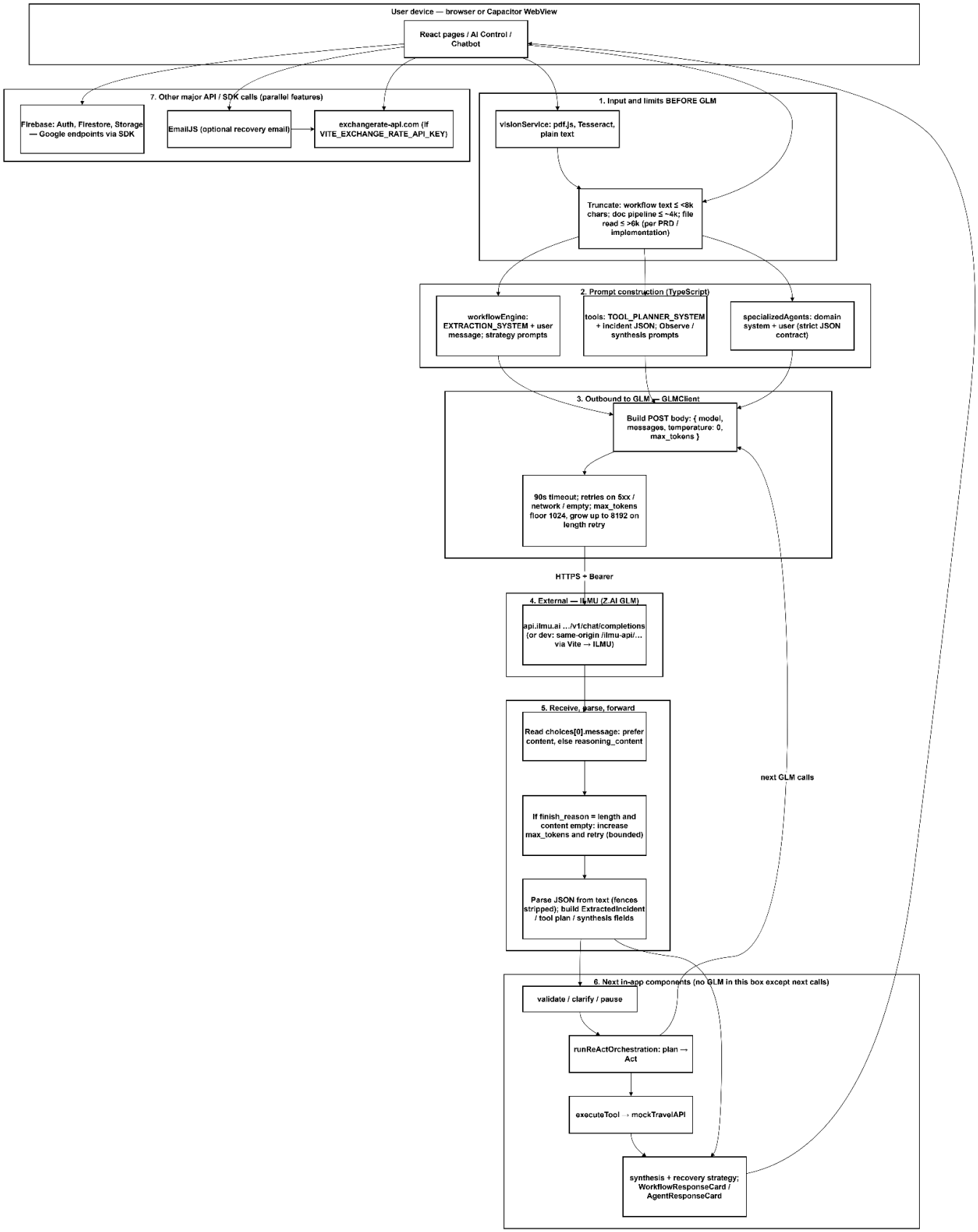
(D) response handling (content vs reasoning_content, finish_reason: length → token budget retry),

(E) parsing and hand-off to the next in-app component,

(F) mock tools vs real HTTP integrations, and

(G) data in Firebase. (Diagram attached below or can view on [Drawio Dependency Diagram ZenTravel])

© UMHackathon 2026

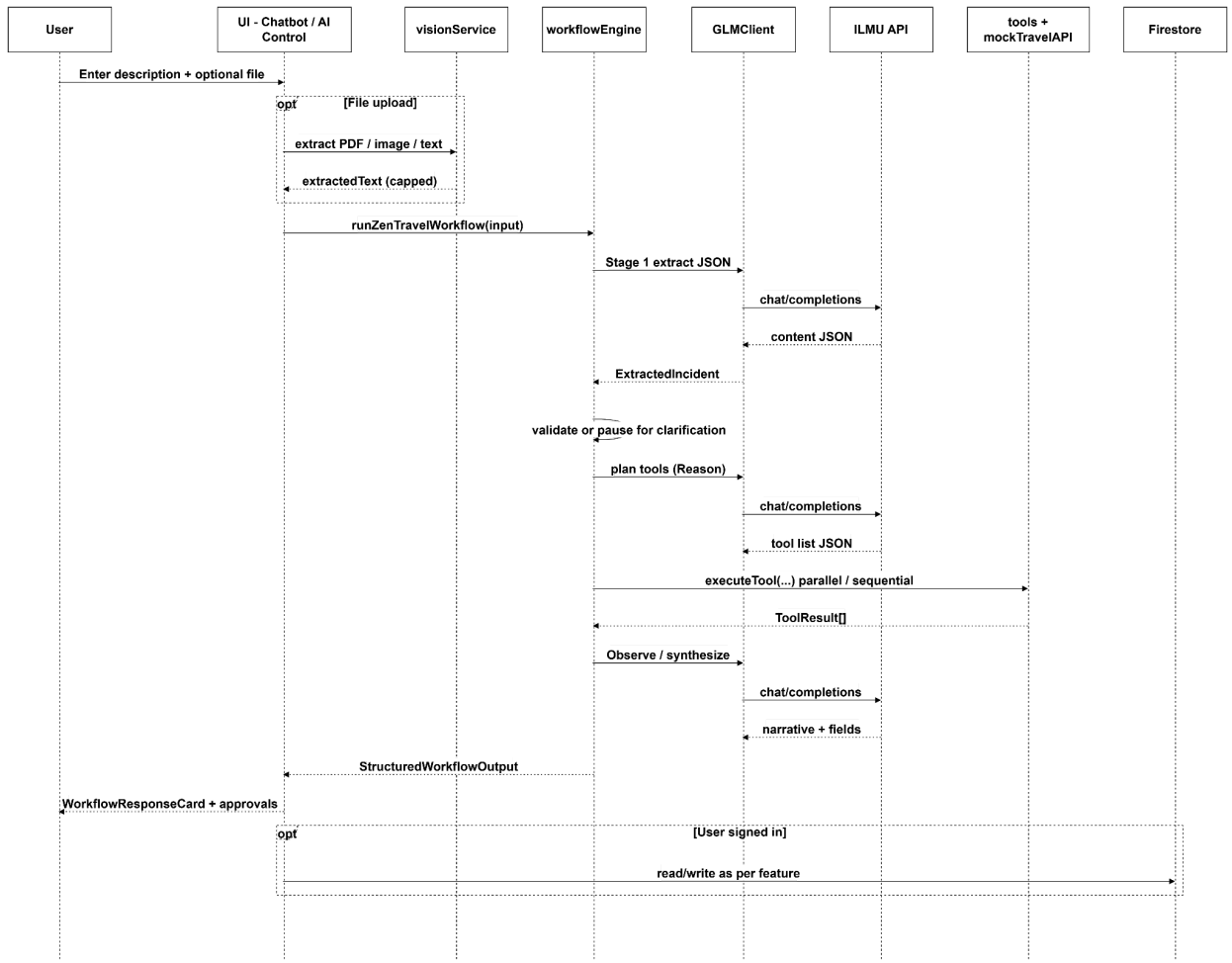


© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2.2. *Sequence Diagram*

Walks one journey: user submits text (optionally after file extraction) through stages to a displayed plan.



([Drawio Sequence Diagram ZenTravel])

2.2. *Technological Stack*

- 2.2.1. ***Frontend:*** *React 19, TypeScript, Vite 8*
- 2.2.2. ***Backend:*** *Firebase (BaaS) for auth, persistence, and storage*
- 2.2.3. ***Database:*** *Cloud Firestore (NoSQL documents)*
- 2.2.4. ***Mobile shell:*** *Capacitor; appId com.zentravel.app; webDir dist*
- 2.2.5. ***LLM:*** *ILMU; model ilmu-glm-5.1 (configurable via VITE_GLM_MODEL)*
- 2.2.6. ***Document/ PDF:*** *pdf.js, Tesseract.js, jsPDF*
- 2.2.7. ***Email:*** *@emailjs/browser*

© UMHackathon 2026

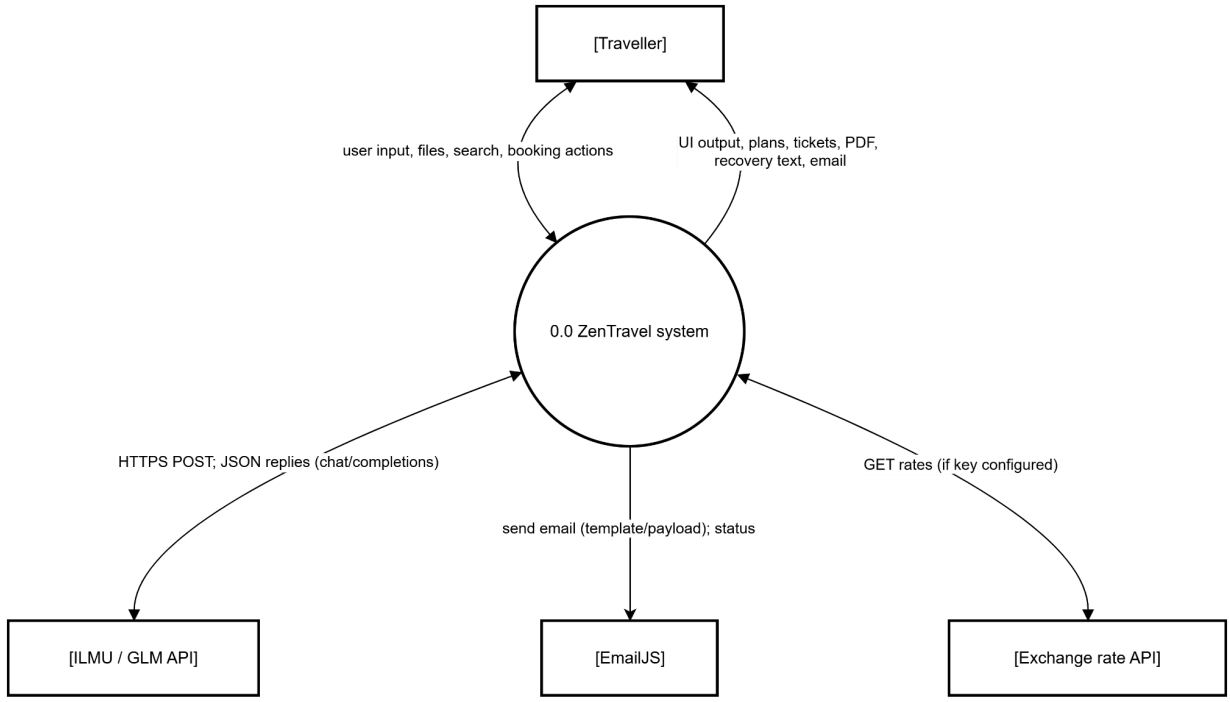
2.3. Key Data Flows

- ***2.3.1. Unauthenticated / light:*** *landing → auth; optional browse; exchange rates if key present.*
- ***2.3.2. Disruption/ AI:*** *user input + files → text caps → GLM + mocks → in-memory + UI; optional email.*
- ***2.3.3. Authenticated:*** *onAuthStateChanged → load users/{uid} for preferences/ cashback; Booking and other collections queried by userId with onSnapshot where implemented.*

2.3.3.1. Data Flow Diagram (DFD)

*DFD "balancing" (no black holes): every input from the user is either **stored in Firestore**, **sent to GLM** (with truncation), **used by mock tools**, or **returned in the UI**; errors are surfaced per PRD (console + user messages).*

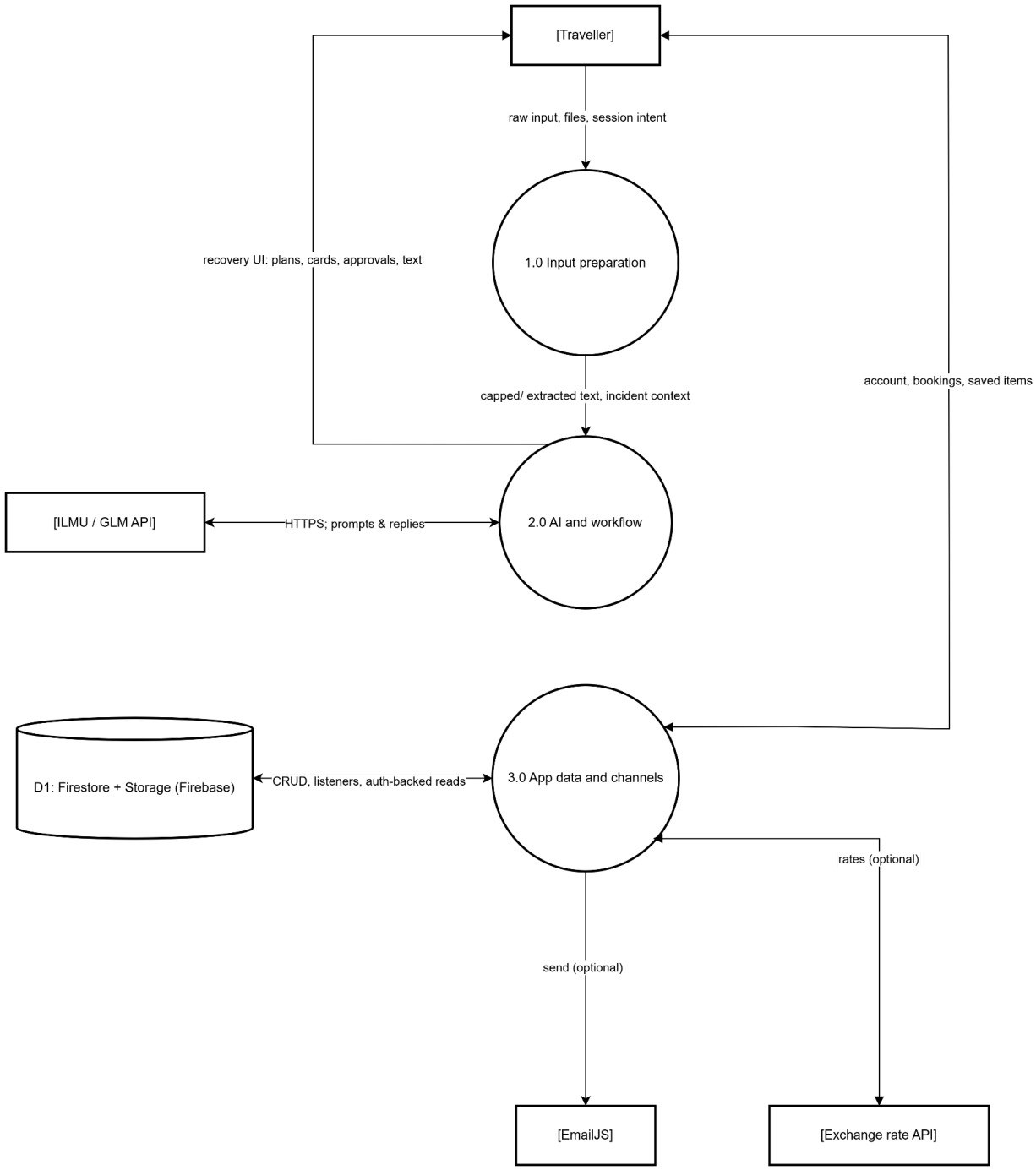
DFD - Level 0 (context)



DFD - Level 1

© UMHackathon 2026

Email: umhackathon@um.edu.my



2.3.4. Normalized Database Schema

a. *Firestore is document-based, not normalized SQL; relationships are by convention (userId fields, bookingNum for tickets).*

Collection/ area	*Primary keys/ access*	*Content (typical)*
users	*Document ID = uid*	*currencyPreference, languagePreference, cashbackBalance, profile fields as implemented*
Booking	*Docs with userId field; ViewTicket uses bookingNum id*	*Booking/ticket data for list and detail views*
search_history	*Per-user via queries*	*Search records from HomePage flows*
favorites	*userId*	*Saved items*
My_Reviews	*userId*	*User reviews*

Note: A full 3NF ERD is not how Firestore is modeled; integrity depends on security rules and application-side validation.

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects.

3.1. Minimum Viable Product:

3.1.1. Core demo features

#	Features	Description
1	*Brain Master workflow*	*Multi-stage pipeline with visible progress, GLM + fallback extraction, ReAct plan, mock tool execution, synthesis and recovery strategy, user approval gating.*
2	*Specialist agents*	*Single-turn (or clarifying) GLM calls with strict JSON outputs for flight, hotel, insurance, trip, and car scenarios.*
3	*Document and image intake*	*PDF text and image OCR feeding the workflow; length caps as per PRD.*
4	*User account and bookings*	*Google sign-in, Firestore sync for profile and Booking list, notifications, refund/ticket views as implemented.*

© UMHackathon 2026

5	*Trip planning + PDF*	*Template/ mock offers, optional GLM narrative, jsPDF export.*

3.2. Non-Functional Requirements (NFRs)

Quality	***Requirements***	***Implementation (as in codebase)***
Reliability of AI path	*When GLM fails, user still gets structured fallback (regex extraction, rule-based tool plan)*	*buildFallbackExtraction, ruleBasedToolPlan, defaults in specializedAgents*
Token/ latency	*GLM calls must not hang indefinitely*	*90 s timeout, 2 retries, loading UI on pages*
Cost efficiency	*Control prompt size and token use*	*Temperature 0, truncation, max_tokens floor and bounded expansion, no extra GLM pre-summary in vision path*
Security awareness	*VITE_* configuration ships in the client build*	*Production hardening can add an API gateway or BFF for secrets and rate limits; current design targets transparent client-side integration*
Cross-platform	*Parity for core flows on web and Capacitor Android*	*Same dist assets; test file upload and WebView behavior separately*

3.3. Out of Scope / Future Enhancements

a. *Live GDS or production Amadeus-class booking.*
b. *Guaranteed legal accuracy of mock compensation rules.*
c. *Chunked RAG for arbitrarily long documents.*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

*4.1.1. **Grayscale Rollout & A/B Testing**:*

© UMHackathon 2026

The team's release strategy is to limit blast radius when shipping changes that affect the whole product, especially the GLM layers (Brain Master prompts, specialist agents, and trip-related AI copy), where latency, quality, and cost are visible to users.

- ***Grayscale (staged) rollout:*** *A new capability (for example: a revised recovery prompt, a new ReAct tool mix, or an AI Control UI change) is first released to a small fraction of real users — on the order of 5% of sessions or install base using Firebase Remote Config (or equivalent feature flags), the Play Console staged rollout for the Capacitor Android package, and/or a canary path for the web dist deployment. The team monitors error rates, ILMU availability and p95 latency, and product signals. If no serious issues are seen within an agreed observation window, the exposure is raised in steps (5% → 25% → 50% → 100%). If regressions appear, the rollout is frozen or reverted at the flag or hosting level before full promotion.*

- ***A/B testing (after a stable grey phase):*** *Once a release has passed a staged rollout, the team runs an A/B test to compare two designs of the same feature. For example, Prompt strategy A vs Prompt strategy B for delay vs cancellation scenarios, or two synthesis templates. Users or requests are randomly assigned to variant A or variant B; the team compares outcomes such as fewer clarification pauses, higher user approval of steps, similar or better time-to-answer. The better-performing variant is adopted as the default in the next general release.*

Current codebase: *the orchestration of percentage rollouts and A/B (Remote Config, store phases, canary) is the intended production design; the repository does not ship a full third-party feature-flag platform by default, so the bullets above are the team's stated operational model for real traffic.*

4.1.2. ***Emergency Rollback (ER) & Golden Release****:*

For recovery and traffic management of ZenTravel, the team defines emergency as a situation where health metrics cross agreed thresholds: for example a sharp rise in ILMU or 5xx errors, a spike in uncaught client errors, or widespread failure of a new GLM or booking-related path. In those cases, Emergency Rollback (ER) is triggered as a deliberate action to protect users.

- ***Emergency rollback (ER)****: The immediate objective is to move users back to a known-good experience: turn off a Remote Config flag, roll back a Play track or static site to the previous deployment, or re-publish a build that the team has already validated in pre-production.*
- ***Golden version****: The Golden Version is the blessed, tagged build (same commit in git, same* `dist`*, same Capacitor output) that was last proven in production; it is kept as a*

labelled release (e.g. `v1.x.x-golden`) and re-deployable without improvising a new build under pressure.

- ***Pipeline**: The intended production setup uses a CI/CD pipeline that can re-deploy the Golden job (repeatable artifact from the golden tag) to web hosting and, where applicable, the app distribution path, so that when ER is triggered, the pipeline enforces: redeploy the Golden Version of the application rather than an ad-hoc fix. In early stages, the team may achieve the same outcome manually (checkout golden tag → npm run build → npx cap sync → republish) until the pipeline is fully automated.*

***Summary:** if anomalies justify ER, the outcome is to restore the Golden Version; automation in the pipeline is the team's target to make that fast and reliable.*

4.1.3. Priority Matrix:

Priority	***Example trigger***	***Example action***
P1	*Firebase or ILMU down for all users*	*Communicate outage; disable AI-heavy routes or show maintenance*
P2	*GLM errors spike*	*Fall back dominate; show disclaimer*

4.2. Monitoring:

***Client:** console warnings in workflow path; user-visible error strings where throw is caught.*

***Future:** error reporting (Sentry, etc.), ILMU **usage dashboards**, Firebase **usage and rules** monitoring.*

4.3. Assumptions:

a. *Users have network access for GLM, Firebase, and optional email/rates APIs.*
b. *VITE_* variables are set for local/dev/demo; missing GLM key → fallback-heavy experience.*
c. *OCR and PDF quality is sufficient for text extraction; poor scans may fail.*

4.4. External Dependencies:

Tools	***Purpose***	***Risks***

© UMHackathon 2026

Z.AI GLM via ILMU	*Extraction, planning, synthesis, agents*	*Rate limits, latency, cost — retries, timeout, fallbacks, truncation*
Firebase (Auth, Firestore)	*Identity, persistence*	*Quotas, rules misconfiguration — test with Firebase console*
EmailJS	*Outbound email from client*	*Template and quota — mailto fallback*
Exchange rate API	*Display conversion*	*Fails soft if no API key*
Capacitor/ Android	*Installed app shell*	*WebView differences for file and storage — device QA*
pdf.js/ Tesseract.js	*Document pipeline*	*CPU on low-end devices; large files slow or truncated*

5. *Project Management & Team Contributions*

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. *Project Timeline:*

Days 1 - 2: *Design architecture and PRD*
Days 3 - 6: *UI designed in Figma, Firebase setup and core functionalities implementation*
Days 7 - 9: *GLM setup, AI system implementation and documentations*
Day 10: *Final checking*

5.2. *Team Members Role:*

Member	**Role**	**Job Details**
TAN SZE HAN	AI Engineer & Intelligence Lead	• Developed the **Z-AI recommendation engine** and intent recognition logic to provide personalized travel suggestions based on user data. • Implemented the *detectIntent* and *extractLocationIntelligence* modules and managed the multi-agent workflow.

© UMHackathon 2026

LIM WAN YEE	Frontend & UI/UX Developer	• Built the core user interface and managed the global state. • Developed the HomePage dynamic views, recommendation modals, and integrated the setView navigation system.
CHAN YI ERN	Full-Stack Security & DevOps Engineer	• Oversee the **Firebase Authentication** system and managed cross-origin security policies for the web and mobile environments. • Secured user sessions using UID matching and handled environment configurations for the Capacitor-based mobile deployment.
NG ROU ROU	Integrations & Financial Systems Specialist	• Managed the integration of external **Exchange Rates APIs** to provide real-time currency conversion for global travel pricing. • Implemented the currency synchronization logic and ensured accurate price rendering for insurance and flight bookings.
KHOR CHEK LIN	Backend & Cloud Database Engineer	• Architected the **Firebase Firestore** NoSQL database and optimized data retrieval for upcoming trips and user history. • Defined the **Booking** and *search_history* schemas and configured composite indexes to resolve complex query errors.

5.3. Recommendations:

For production:

- *Consider a BFF or API gateway to centralise rate limits, observability, and key handling*
- *Add Sentry*
- *Expand Firestore security rules tests*
- *Add E2E (Playwright) for main flows*
- *Cache exchange rates behind a small service if traffic grows*

© UMHackathon 2026